SLE-4

ARCHITECTURE DECISION RECORD (ADR)

BFS vs DFS Graph Search System

Name	Aditi Annaso Dudhal
PRN	25UAM134
Division	B
Course	02AML204 – Introduction to Artificial Intelligence
Status	Accepted

Prepared from the BFS/DFS profiling and C4 architecture work

PART A: ARCHITECTURE DECISION RECORD (ADR)

Title: Compare BFS and DFS Using the Same Graph and Profiling Conditions

Status: Accepted

1. Context

The BFS vs DFS Graph Search System implements Breadth-First Search (BFS) and Depth-First Search (DFS) to search a graph. BFS explores nodes level by level using a queue, while DFS explores deeper along a path before backtracking, using a stack or recursion. In SLE-2, the algorithms are implemented and profiled so their behaviour can be compared. The project setup uses starting node A and goal nodes B, I, and K, with profiling scripts designed for 10 timing runs and summaries of best, average, and worst execution time. SLE-3 represents the system using C4 context, container, and component diagrams.

2. Decision

I chose to keep BFS and DFS as separate implementations and evaluate them using the same graph, starting node A, and selected goal nodes B, I, and K. Their profiling scripts are used to collect repeated timing measurements under consistent conditions. Conclusions about performance will be based on actual recorded outputs, not assumed values.

3. Alternatives Considered

- **Alternative 1: Implement only BFS.** This demonstrates level-order graph search but does not allow a direct comparison with DFS.
- **Alternative 2: Implement only DFS.** This demonstrates depth-first exploration but does not show how its behaviour differs from BFS.
- **Alternative 3: Profile the algorithms with different inputs.** Different graph inputs or start/goal conditions could make the performance comparison less meaningful.
- **Alternative 4: Compare only theoretical complexity.** Theory is useful, but profiling provides observed runtime information for the selected test cases.

4. Rationale / Justification

Keeping the algorithms separate makes their implementations easier to run, inspect, and test. Using the same graph and search conditions reduces differences caused by the inputs, so the observed results are more suitable for comparison. Repeated timing runs and best/average/worst summaries help show variation in measured execution time. The profiling scripts and result files are important evidence for the decision; actual values should be copied from the scripts' output before making a claim about which algorithm was faster. This decision also matches the SLE-3 C4 design, which separates the overall system, its containers, and the internal search/profiling components.

5. Consequences

Positive: BFS and DFS can be examined independently; shared test conditions make the comparison easier to repeat; profiling output supports evidence-based discussion; and the separate scripts and architecture diagrams make the system easier to explain and maintain.

Negative / Trade-offs: Timing can vary by computer and runtime environment. A small set of goal nodes cannot represent every graph-search case. Execution time alone does not measure every aspect of performance, and the profiling results must be checked before drawing conclusions.

6. Validation

Verify that both algorithms use the intended graph and start node A; confirm the goal-node cases B, I, and K; run the profiling scripts; and check that the reported timing summaries come from actual runs. Save the profiling output for review. Confirm that the SLE-3 C4 context, container, and component diagrams describe the same BFS/DFS system. No numerical performance result is included here because verified profiling output was not available while preparing this report.

PART B: SHORT SUMMARY OF MY SLE JOURNEY

SLE-1 – AI-Augmented Workflow: I worked on a basic AI agent and documented AI-assisted development in an AI Contribution Log, with responsibility for reviewing the work.

SLE-2 – BFS/DFS Implementation and Profiling: I worked on the BFS vs DFS Graph Search System and its profiling scripts. The setup uses starting node A and goal nodes B, I, and K; the scripts are intended to perform repeated timing runs and summarize best, average, and worst times.

SLE-3 – C4 Architectural Design: I prepared the system's context, container, and component diagrams to describe its scope, main parts, and internal responsibilities.

SLE-4 – Architecture Decision Record: I documented the decision to compare BFS and DFS using the same graph and test conditions, including alternatives, justification, consequences, and validation.

PART C: AI CONTRIBUTION

AI assistance was used to help organize the ADR sections, clarify the BFS/DFS design reasoning, and improve the structure of the report. I am responsible for reviewing the code and diagrams in my repositories, running the profiling scripts, verifying the actual measurements, and ensuring the final report accurately reflects the implemented project. Any claim that does not match the repository or its output must be corrected.

Conclusion

The main decision in this SLE-4 report is to keep BFS and DFS as separate implementations and compare them under the same graph and test conditions. This makes the profiling exercise easier to repeat and interpret. SLE-2 provides the implementation and profiling work, while SLE-3 documents the system architecture using the C4 model. Final performance conclusions must be supported by verified profiling output.

References

SLE-2: BFS and DFS repository

https://github.com/juishinde136-arch/IAI_SLE2_BFS_DFS

SLE-3: C4 architecture repository

https://github.com/Aditidudhal/IAI-SLE/tree/main/SLE3